Trabalho 1 — Processos vs. Threads

Arthur Pinto Ferreira, Matheus Corbellini Steindorff, Rafaela do Nascimento Mello, Vinicius Moço Quintian

Escola Politécnica — Pontifícia Universidade Católica do Rio Grande do Sul (PUCRS)

Resumo. Este trabalho compara o overhead de criação, o custo de comunicação e a consistência de dados entre processos (POSIX fork) e threads (POSIX pthreads) em ambiente Linux. O experimento consiste em incrementar um contador global até 1.000.000.000, distribuindo o trabalho entre N unidades de execução (N = 2, 4 e 8), em quatro cenários: threads sem mutex (T1), threads com pthread_mutex (T2), processos com memória compartilhada sem semáforo (P1) e processos com memória compartilhada e sem_open (P2). Os resultados mostram que threads possuem menor overhead de criação e comunicação, enquanto processos com semáforos POSIX nomeados apresentam o maior custo de sincronização. Sem mecanismos de sincronização, ambos os modelos produzem corrupção de dados por condição de corrida.

1. Introdução

Este trabalho compara o overhead de criação, o custo de comunicação e a consistência de dados entre processos (POSIX fork) e threads (POSIX pthreads) em ambiente Linux.

O experimento consiste em incrementar um contador global até 1.000.000.000 (um bilhão), distribuindo o trabalho entre N unidades de execução (N = 2, 4 e 8), em quatro cenários. Cada cenário foi executado com N = 2, 4 e 8 unidades de execução, totalizando 12 combinações distintas. Os resultados foram analisados em termos de tempo de execução, valor final do contador e consistência dos dados.

Código	Descrição
T1	Threads sem mutex (sem sincronização)
T2	Threads com pthread_mutex
P1	Processos via fork + memória compartilhada (shmget) sem semáforo
P2	Processos via fork + memória compartilhada com sem_open

2. Desenvolvimento

2.1. Assinatura do Hardware

O processador utilizado é um Intel Core i3-N305 com 8 núcleos físicos e 1 thread por núcleo (sem Hyper-Threading). Isso significa que os experimentos com N = 8 podem saturar todos os núcleos disponíveis, o que impacta diretamente no comportamento de concorrência e na frequência de colisões observadas nos experimentos sem sincronização.

Cada núcleo possui seu próprio cache L1d (256 KiB) e L1i (512 KiB), além de cache L2 privado de 16 MiB por instância. O cache L3 de 6 MiB é compartilhado entre todos os núcleos. O ambiente é virtualizado sob Microsoft Hyper-V com suporte a VT-x, sobre arquitetura x86_64 com Byte Order Little Endian.

2.2. Resultados

A Tabela 1 apresenta os resultados completos dos experimentos, incluindo tempo de execução e valor final do contador para cada cenário e número de trabalhadores. Os experimentos com sincronização (T2 e P2) sempre atingem o valor correto de 1.000.000.000, enquanto os sem sincronização (T1 e P1) apresentam corrupção crescente conforme N aumenta.

Tabela 1. Resultados completos — Tempo de execução e Valor Final do contador

Experimento	N	Tempo (s)	Valor Final
T1 — threads sem mutex	2	3,3047 s	654.290.134
T1 — threads sem mutex	4	7,5468 s	684.678.724
T1 — threads sem mutex	8	17,6189 s	447.515.488
T2 — threads com mutex	2	99,6028 s	1.000.000.000
T2 — threads com mutex	4	93,5196 s	1.000.000.000
T2 — threads com mutex	8	90,7124 s	1.000.000.000
P1 — processos sem semáforo	2	2,5497 s	533.402.362
P1 — processos sem semáforo	4	2,8438 s	319.062.540
P1 — processos sem semáforo	8	3,1545 s	203.808.717
P2 — processos com semáforo	2	182,4934 s	1.000.000.000
P2 — processos com semáforo	4	200,6238 s	1.000.000.000
P2 — processos com semáforo	8	303,9470 s	1.000.000.000

2.3. Gráfico 1 — Tempo de Execução vs. Número de Trabalhadores (N)

O gráfico de linhas mostra o tempo de execução (em segundos) para cada cenário conforme N varia entre 2, 4 e 8 trabalhadores. Os cenários com sincronização (T2 e P2) dominam o eixo Y: P2 sobe de ~182 s (N=2) até ~304 s (N=8), pois cada incremento exige uma chamada de sistema de semáforo, que escala pessimamente com mais processos concorrendo pelo mesmo recurso. T2 apresenta comportamento levemente decrescente (~99 s → ~91 s), pois o mutex de thread tem overhead fixo mais baixo e a contenção se distribui melhor entre os núcleos. Os cenários sem sincronização (T1 e P1) ficam próximos de zero na escala do gráfico: P1 mal ultrapassa 3 s mesmo em N=8, e T1 chega a ~18 s em N=8 — tempos baixos que resultam diretamente da corrupção de dados (menos incrementos efetivos sendo contabilizados).

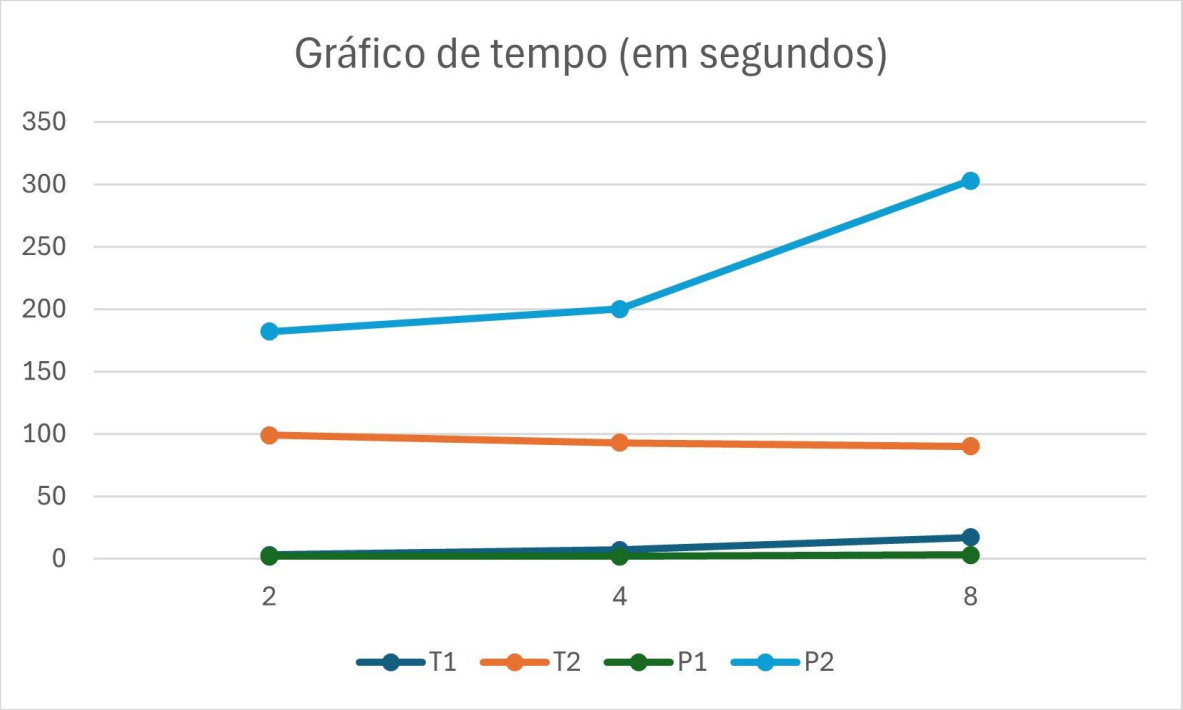

Figura 1. Gráfico de Tempo de Execução (em segundos) vs. Número de Trabalhadores (N)

2.4. Gráfico 2 — Valor Final do Contador vs. Número de Trabalhadores (N)

O gráfico de barras compara o valor final alcançado pelo contador em cada cenário e valor de N. T2 e P2 (com sincronização) atingem consistentemente 1.000.000.000 em todos os valores de N, confirmando que os mecanismos de exclusão mútua garantem a integridade dos dados. T1 e P1 (sem sincronização) apresentam valores muito abaixo de 1 bilhão, com degradação crescente conforme N aumenta: P1 cai de ~533 milhões (N=2) para apenas ~204 milhões (N=8), evidenciando que mais núcleos paralelos amplificam as colisões de leitura/escrita na memória compartilhada. T1 oscila entre ~447 e ~685 milhões, com comportamento menos previsível devido à forma como o escalonador distribui as threads entre os núcleos.

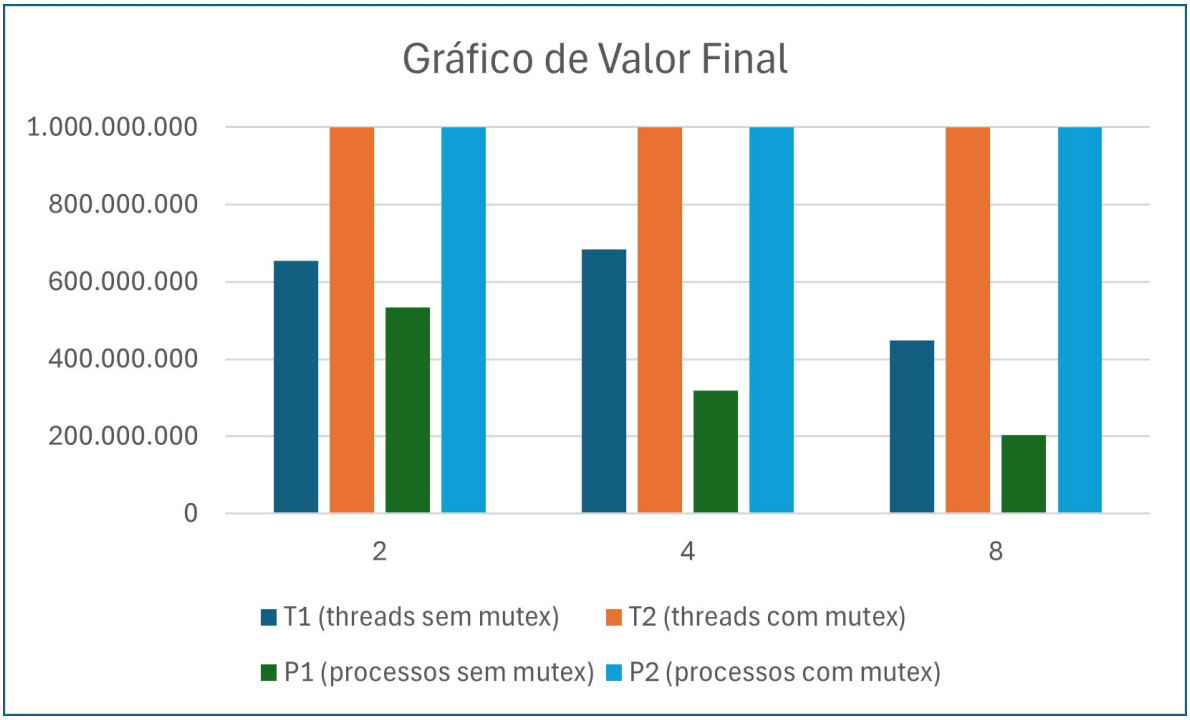

Figura 2. Gráfico de Valor Final do Contador vs. Número de Trabalhadores (N)

2.5. Análise de Corrupção

Nos experimentos T1 (threads sem mutex) e P1 (processos sem semáforo), o contador não atingiu 1.000.000.000. A operação contador++ não é atômica. Em nível de máquina ela se decompõe em três etapas: leitura do valor atual, incremento em registrador e escrita do resultado. Quando múltiplas unidades de execução realizam essas etapas simultaneamente sem sincronização, ocorre uma condição de corrida (race condition):

1. Unidade A lê contador = 1000.
2. Unidade B lê contador = 1000 (antes de A escrever).
3. A escreve contador = 1001.
4. B escreve contador = 1001 — o incremento de A foi sobrescrito e perdido.

Esse fenômeno é chamado de lost update (atualização perdida).

O i3-N305 possui 8 núcleos independentes sem Hyper-Threading, o que significa que com N = 8 todos os núcleos executam genuinamente em paralelo, sem compartilhamento de pipeline. Isso maximiza as janelas de colisão entre as leituras e escritas concorrentes, explicando por que P1 com N = 8 (203.808.717) teve o pior índice de corrupção — quase 80% das incrementações foram perdidas. Cada núcleo possui seu próprio cache L1/L2 privado, o que agrava a inconsistência: cada núcleo pode ter uma cópia diferente do contador em seu cache antes de propagá-la para o L3 compartilhado.

Tabela 2. Valores finais com corrupção (abaixo de 1 bilhão)

Experimento	N = 2	N = 4	N = 8
T1 — threads sem mutex	654.290.134	684.678.724	447.515.488
P1 — processos sem semáforo	533.402.362	319.062.540	203.808.717

3. Conclusão

3.1. Overhead de Criação

Criar um processo via fork tem custo maior que criar uma thread: o sistema operacional precisa duplicar o espaço de endereçamento do processo pai (copy-on-write), alocar uma entrada na tabela de processos e configurar canais de IPC. Threads compartilham o mesmo espaço de endereçamento do processo criador, tornando sua criação significativamente mais leve.

Nos experimentos sem sincronização, P1 foi mais rápido que T1 em todos os cenários, o que pode parecer contraditório. Porém, isso se deve à menor corrupção de cache entre processos no início da execução — cada processo tem espaço próprio — e não indica menor overhead de criação.

3.2. Comunicação

Threads compartilham memória automaticamente via variáveis globais. A comunicação é implícita e de baixo custo, mas exige sincronização explícita para evitar race conditions. Processos precisam de mecanismos explícitos de IPC como memória compartilhada (shmget/shmat), com custo adicional de configuração e de chamadas de sistema, além da necessidade de semáforos para proteger o acesso. Threads são claramente mais eficientes na comunicação para dados compartilhados.

3.3. Consistência de Dados

Sem mecanismos de sincronização (mutex ou semáforo), nenhum modelo garante consistência. Com sincronização, ambos atingem o resultado correto, porém com alto custo de tempo — especialmente P2 (semáforos POSIX nomeados têm overhead de chamada de sistema maior que mutexes de threads).

Tabela 3. Consistência dos dados por cenário (N = 8)

Cenário	Resultado com N = 8	Consistente?
T1	447.515.488	Não
T2	1.000.000.000	Sim
P1	203.808.717	Não

P2	1.000.000.000	Sim

3.4. Comparativo Final

Tabela 4. Comparativo entre Threads (pthreads) e Processos (fork)

Aspecto	Threads (pthreads)	Processos (fork)
Overhead de criação	Baixo	Alto
Comunicação	Eficiente (memória compartilhada nativa)	Custosa (IPC explícita via shmget)
Consistência sem sinc.	Corrompida	Corrompida (mais severa com mais núcleos)
Consistência com sinc.	Sim (mutex) — ~95 s	Sim (semáforo) — 180–304 s

Processos têm maior overhead de criação e de comunicação. Threads são mais eficientes para tarefas que compartilham dados, desde que a sincronização seja aplicada corretamente.